

Andropedia Website — Backend Audit & PRD

Status	Draft v2 - Phases 1-3 implemented on branches <code>backend/phase-1-recruitment</code> , <code>backend/phase-2-database</code> , <code>backend/phase-3-auth</code> . Phase 3 used emailed one-time-code login with database sessions instead of Supabase Auth (see section 2.5 note)
Date	2026-10-06
v2 change	Recruitment applications go to a Google Sheet (via Google Form or the Sheets API) with CSV export, not the app database. Admins build the recruitment dashboard separately. Every applicant gets a confirmation email. See §2.7.1.
Scope	Backend: data storage, auth, API routes, forms, admin tooling
Audited branch	<code>main @ 16c0a34</code> (same as <code>upstream/main</code>), plus <code>upstream/ui-updated @ d876d94</code>

Part 1 — Audit: What's Broken Today

Summary: The site has a complete-looking frontend, but its backend is a demo. The site stores no data permanently, the server never checks who you are, and several forms look like they work but send nothing anywhere. Prisma and a Postgres schema are in the repo but are never used.

Severity: ● Critical (data loss or security hole) · ● High (feature doesn't work) · ● Medium (wrong or misleading behaviour)

1. Database / persistence

#	Severity	Problem	Evidence
D1	●	All data lives in in-memory arrays and is lost on every restart. On Vercel (serverless) it isn't even shared between requests.	<code>src/lib/data-store.ts:326-352</code>
D2	●	The Prisma client is created but never imported anywhere. Every API route reads from <code>data-store.ts</code> .	<code>src/lib/prisma.ts</code> , all of <code>src/app/api/**</code>
D3	●	No <code>.env</code> and no connection strings. <code>prisma.config.ts</code> expects <code>DIRECT_URL</code> , and nobody has confirmed a Supabase project exists.	<code>prisma.config.ts:10</code> , <code>.gitignore (.env*)</code>
D4	●	No migrations (<code>prisma/migrations/</code> doesn't exist), so no tables exist in any Postgres database.	—
D5	●	<code>PrismaClient</code> is constructed with no driver adapter. Prisma 7 expects one (e.g. <code>@prisma/adapter-pg</code>); check before going live.	<code>src/lib/prisma.ts:9</code>
D6	●	The schema lacks models for recruitment applications, events/RSVPs and sessions. The SQLite <code>dev.db</code> on <code>ui-updated</code> has those tables, but the Postgres schema on <code>main</code> doesn't.	<code>prisma/schema.prisma</code>
D7	●	Seed data (fake users, weeks, tasks) is hardcoded in the code instead of in a seed script.	<code>src/lib/data-store.ts:4-324</code>

2. Authentication & authorization

#	Severity	Problem	Evidence
A1	●	The password is never checked. The login form has a pre-filled <code>password123</code> but only sends the email. Anyone who knows a listed email can log in.	<code>src/app/portal/login/page.tsx:13,22</code> , <code>src/app/api/auth/login/route.ts</code>
A2	●	The session token is fake: <code>demo_token_<id>_<timestamp></code> . It isn't signed, never expires, and no route checks it.	<code>src/app/api/auth/login/route.ts:27</code>
A3	●	The logged-in user is stored in <code>localStorage</code> and trusted as-is. Editing <code>role</code> to <code>super_admin</code> in devtools gives full admin access.	<code>src/lib/auth-context.tsx</code> , <code>src/components/portal/PortalAccessGate.tsx</code>
A4	●	Role checks happen only in the browser. The server never checks who is calling.	<code>PortalAccessGate.tsx</code>
A5	●	Real members (from the Google Sheet) cannot log in . Login only searches the hardcoded seed users, not the sheet members.	<code>api/auth/login/route.ts:9</code> vs <code>src/lib/live-members.ts</code>
A6	●	No sign-up or onboarding, no password reset, no server-side logout or session revocation.	—
A7	●	The dashboard falls back to <code>usr_1</code> when no user is set, so it can show another person's tasks.	<code>src/app/portal/dashboard/page.tsx:31</code>

3. API security & validation

#	Severity	Problem	Evidence
S1	●	POST <code>/api/evaluations</code> needs no login. Anyone can grade any task and award points. <code>adminId</code> / <code>adminName</code> come from the request body, so they can be faked.	<code>src/app/api/evaluations/route.ts</code>
S2	●	POST <code>/api/tasks</code> needs no login. Anyone can submit as any <code>userId</code> and set any <code>userName</code> / <code>userAvatar</code> .	<code>src/app/api/tasks/route.ts:32</code>
S3	●	GET <code>/api/members</code> returns the email address of every member , from the seed data and the Google Sheet, to anyone on the internet.	<code>src/app/api/members/route.ts</code> , <code>src/lib/live-members.ts</code>
S4	●	GET <code>/api/tasks</code> returns every submission and its private feedback to anyone.	<code>src/app/api/tasks/route.ts:4</code>
S5	●	Submissions to a closed week aren't blocked. <code>weekId</code> isn't checked against existing weeks.	<code>src/app/api/tasks/route.ts</code>
S6	●	A domain lead can grade tasks from other domains.	<code>src/app/api/evaluations/route.ts</code>
S7	●	No checks on URLs, text length, <code>domain</code> values or rubric sub-scores (each 0–25, summing to <code>score</code>).	<code>api/tasks</code> , <code>api/evaluations</code>
S8	●	No rate limiting on login, submission or recruitment endpoints.	—

4. Forms that show success but save nothing

#	Severity	Problem	Evidence
F1	●	Recruitment applications are thrown away. On <code>main</code> the form just sets <code>submitted=true</code> and never sends the data. On <code>ui-updated</code> it reaches <code>POST /api/recruitment</code> but is only stored in memory. It can't be viewed and is lost on restart.	<code>src/app/join/page.tsx:63</code> ; <code>ui-updated: src/app/api/recruitment/route.ts</code>
F2	●	Event "Reserve Seat" only updates the page in the browser. No RSVP is stored and seat limits aren't enforced.	<code>src/app/events/page.tsx:88</code>
F3	●	The admin "Open/Close week" toggle only updates the page and resets on reload. There's no API to save it.	<code>src/app/portal/admin/page.tsx:48</code>

5. Business-logic bugs

#	Severity	Problem	Evidence
L1	●	Re-grading an already-graded task adds its score to the member's points again , so points are double-counted.	<code>src/lib/data-store.ts:389-393</code>
L2	●	The leaderboard's <code>rankChange</code> is <code>Math.random()</code> , so the ▲/▼ arrows are fake.	<code>src/lib/data-store.ts:442</code>
L3	●	The <code>period</code> filter ("Current Week", "Monthly") is accepted but ignored, so it always shows all-time results.	<code>getLeaderboard()</code>
L4	●	Score = real grades + <code>points × 0.7</code> from hardcoded seed values, which mixes real and fake numbers.	<code>src/lib/data-store.ts:420</code>
L5	●	<code>streakWeeks</code> is hardcoded and never calculated. Badges depend on domain, not on what the member achieved.	<code>getLeaderboard()</code>

6. Missing admin and content management

#	Severity	Problem
M1	●	No way to create, edit or delete weeks/sprints. They're hardcoded.
M2	●	No way to add members, change roles or deactivate accounts. Staff are hardcoded and members come from a public Google Sheet.
M3	●	No admin view for recruitment applications or event RSVPs. (<i>Recruitment: the admins will build a separate dashboard from the Google Sheet / CSV; see §2.7.1.</i>)
M4	●	Events and projects are hardcoded in page components (or <code>src/content/*</code> on <code>ui-updated</code>), so changing them needs a code change.

7. Platform and operations

#	Severity	Problem
O1	●	No email notifications (application received, task graded, RSVP confirmed).
O2	●	No file uploads. Submissions are links only, and member photos go through Google Drive thumbnails.
O3	●	No <code>.env.example</code> , no documented setup for environment variables.

#	Severity	Problem
O4	●	No automated tests, no error tracking, no audit log of admin actions.
O5	●	main and ui-updated have diverged a lot. ui-updated nests the app in Andropedia-website-main/ and has a better recruitment flow and public-profile sanitising.

Part 2 — PRD: Production Backend

2.1 Problem statement

Andropedia runs weekly sprints, evaluations, a leaderboard, events and recruitment through this website, but none of that data is saved and none of it is secured. Applicants get a success message while their application is thrown away. Anyone can make themselves an admin or award points. The club can't run a real recruitment cycle or sprint season on the current system.

2.2 Goals

1. **Keep every piece of user-submitted data permanently:** applications, task submissions, evaluations, RSVPs and admin settings.
2. **Real identity:** only verified club members can log in, and every protected API checks the session and role on the server.
3. **Correct numbers:** leaderboard, points and streaks are calculated from real evaluation data.
4. **Admins run the club without code changes:** manage weeks, members, roles, events and applications from the portal.
5. **Protect personal data:** no public endpoint exposes emails or private feedback.

2.3 Non-goals (this release)

- Native mobile app
- Payments or ticketing
- In-app file storage for submission files (links stay; uploads come later)
- Real-time features (WebSockets, live leaderboard push)
- **Recruitment admin dashboard.** Admins will build it separately from the Google Sheet / CSV. This project only has to deliver applications into the sheet in a stable format (\$2.7.1).

2.4 Users & roles

Role	Can do
Public visitor	View public pages, submit a recruitment application, RSVP to public events
Member	Everything above, plus submit tasks for open weeks, view own tasks and feedback, view leaderboard
Domain admin (lead)	Everything above, plus view and grade submissions in their own domain , view applications for their domain
Super admin	Everything, plus manage weeks, members, roles, events and all applications; export data

2.5 Recommended architecture

Concern	Recommendation	Why
Database	Supabase Postgres via Prisma (already in the repo)	Prisma and the Supabase config are already set up; free tier is enough for a club
Auth	Supabase Auth with email magic link / OTP, limited to the college email domain	No passwords to store; ties identity to college email; works server-side with <code>@supabase/ssr</code>
Session	HTTP-only cookie session read on the server in every API route and server component	Replaces the <code>localStorage</code> user (A2–A4)
Authorization	One <code>requireUser(role?, domain?)</code> helper used by every protected route	One place for role and domain checks
Validation	Zod schemas shared by forms and API routes	Fixes S7; one source of truth for input rules
Recruitment storage	Google Sheet (written by the Sheets API, or through a Google Form), exported as CSV for the dashboard	Admins already work in Google tools; keeps recruitment independent of the app database; see §2.7.1
Email	Resend, or Gmail SMTP via Nodemailer, for transactional mail	Recruitment confirmations (every applicant) and later notifications
Rate limiting	Upstash Ratelimit (or a simple Postgres-backed limiter) on login, recruitment, RSVP	Fixes S8

Alternative if Supabase isn't wanted: Neon Postgres + Auth.js (NextAuth) with an email provider. The PRD otherwise stays the same.

2.6 Data model changes

Keep the existing `User`, `Week`, `Task` and `Evaluation` models, and add the models below. Recruitment applications are **not** stored in Postgres; they go to the Google Sheet (§2.7.1).

```

model Event {
  id          String   @id @default(cuid())
  title       String
  description  String
  startsAt    DateTime
  location     String?
  capacity     Int?
  isPublished Boolean @default(false)
  rsvps       Rsvp[]
}

model Rsvp {
  id          String   @id @default(cuid())
  eventId     String
  event       Event    @relation(fields: [eventId], references: [id], onDelete: Cascade)
  name        String
  email       String
  createdAt   DateTime @default(now())
  @@unique([eventId, email])
}

model AuditLog {
  id          String   @id @default(cuid())
  actorId     String
  action      String    // e.g. "evaluation.create", "week.toggle", "user.role.update"
  target      String
  meta        Json?
  createdAt   DateTime @default(now())
}

```

Changes to existing models:

- **User**: add `authId String? @unique` (link to Supabase Auth user) and `isActive Boolean @default(true)`. Remove the stored totals `points` / `tasksCompleted` / `streakWeeks`, or keep them only as a cache that's recalculated (fixes L1, L4, L5).
- **Task**: add `@@unique([userId, weekId])` so each member submits once per week (re-submitting edits the existing task).
- **Evaluation**: keep `taskId @unique`. Re-grading **updates** the existing row instead of adding points (fixes L1).

2.7 Functional requirements

P0 — Must ship before the next recruitment / sprint cycle

ID	Requirement	Fixes
R1	Create the Supabase project, add <code>DATABASE_URL</code> + <code>DIRECT_URL</code> , add <code>.env.example</code> , run the first migration.	D3, D4, O3
R2	Replace every function in <code>data-store.ts</code> with Prisma queries; move the sample data into <code>prisma/seed.ts</code> .	D1, D2, D7
R3	Recruitment : bring over the validated <code>POST /api/recruitment</code> and <code>RecruitmentForm</code> from <code>ui-updated</code> ; add each application as a row in the recruitment Google Sheet; return the reference ID; send a confirmation email to every applicant ; rate-limit and spam-protect. Needs no database or auth work, so it can ship first. Full spec in §2.7.1.	F1

ID	Requirement	Fixes
R4	Auth: Supabase Auth magic-link login limited to the college domain; HTTP-only session cookie; delete the <code>demo_token</code> and the trust in <code>localStorage</code> ; the browser gets the user from a <code>GET /api/me</code> call.	A1–A4, A6
R5	Server-side authorization on every mutating route via <code>requireUser()</code> . <code>userId</code> / <code>adminId</code> come from the session, never from the request body .	S1, S2, A4
R6	Remove PII from public endpoints: <code>GET /api/members</code> returns public profile fields only (port <code>toPublicMemberProfile</code> from <code>ui-updated</code>). <code>GET /api/tasks</code> requires login and is scoped to the caller's own tasks (member) or their domain (lead).	S3, S4
R7	Member import: a one-off script plus admin action that imports Google Sheet members into <code>User</code> so real members can log in.	A5, M2

P1 — Core admin and correctness

ID	Requirement	Fixes
R8	Week management API + UI: create, edit, open/close. Only one week can be active. Task submission is rejected when the week is closed.	F3, M1, S5
R9	Evaluation rules: leads may only grade their own domain; rubric sub-scores checked (0–25 each, sum = score); re-grading updates the existing evaluation. Every grade is written to <code>AuditLog</code> .	S6, S7, L1
R10	Leaderboard calculated from <code>Evaluation</code> rows with real <code>period</code> filters (current week, last 30 days, all-time); <code>rankChange</code> compared with the previous week's ranking; streak = consecutive weeks with an evaluated task.	L2–L5
R11	Recruitment admin view. Out of scope. Admins build a separate dashboard from the Google Sheet / CSV. This project only guarantees the sheet's column format (§2.7.1).	M3
R12	Member and role management: invite, change role/domain, deactivate.	M2
R13	Zod validation on all request bodies; consistent error format <code>{ success: false, error, fieldErrors? }</code> .	S7

P2 — Nice to have

ID	Requirement	Fixes
R14	Events from the DB with RSVP API, capacity enforcement, duplicate protection, confirmation email, admin attendee list.	F2, M4
R15	Email notifications: task graded, application status changed.	O1
R16	Projects managed from the DB instead of hardcoded content.	M4
R17	File uploads (Supabase Storage) for submissions and avatars.	O2
R18	Error tracking (Sentry) and API tests for auth, recruitment, tasks and evaluations.	O4

2.7.1 Recruitment pipeline (R3 in detail)

Decision: applications go into a **Google Sheet**. Admins get a **CSV** from that sheet and build their own dashboard on it. The website never stores applications itself, and every applicant gets a confirmation email.

Flow

```
Applicant fills /join form
  |
  ▼
POST /api/recruitment
  1. Check spam trap, rate limits, inputs (Zod)
  2. Generate reference REC-XXXXXXX
  3. Add row to Google Sheet — fails —> 503 "Couldn't submit, please try again"
    | ok                                (no email sent, no success screen)
    ▼
  4. Send confirmation email — fails —> still 201; row marked email_status = failed
    | ok                                (resent later by retry script)
    ▼
  5. Update row email_status = sent
  6. 201 { success: true, reference }
    |
    ▼
Success screen shows reference ID

Admins' dashboard (separate project) ◀— reads Google Sheet / CSV export
```

Rule: the applicant sees a success screen **only after the row is confirmed written to the sheet.**

How applications get into Google

Option	How it works	Pros	Cons
A. Google Sheets API (<i>recommended</i>)	A Google Cloud service account, given edit access to the sheet, adds rows (<code>spreadsheets.values.append</code>) from the API route	Official, documented API; confirms each write; can record <code>email_status</code> ; full control of columns	One-time Google Cloud setup; service-account key stored as a secret environment variable
B. Google Form (formResponse)	The API route forwards the fields to the form's hidden <code>entry.<id></code> inputs; responses show in Google Forms and its linked sheet	No credentials; admins also get Google Forms' built-in summary charts	Undocumented endpoint that can break without notice; can't reliably confirm the save; breaks if questions are edited (field IDs change) or the form requires sign-in; can't record <code>email_status</code>
C. CSV file written by the server	Rows added to a <code>.csv</code> file	—	Not viable on Vercel: the server's filesystem is read-only and doesn't keep files between requests. Using external file storage instead risks losing rows when two people submit at once.

Recommendation: use Option A. The CSV comes from the sheet itself: *File* → *Download* → *CSV*, or the sheet's `.../export?format=csv&gid=<tab-id>` URL for an admin dashboard (the sheet must stay private, so the dashboard needs Google access). If admins want the Google Forms interface, Option B is acceptable, but then the success screen can't be fully trusted (see the Rule above).

Sheet column contract (what the dashboard can rely on)

v1 recruitment page: the home page only shows the domain descriptions and a **Join Now** button; applying happens on the dedicated `/join` page (overview, who can apply, selection steps, a 4-step form with a domain-specific question, FAQ). Its text lives in `src/content/recruitment.ts`.

The website **only adds rows**. It never edits, reorders or deletes existing columns or rows. New columns are only ever added at the right-hand end.

Col	Header	Example	Notes
A	reference	REC-1A2B3C4D	Unique; also in the email
B	submitted_at	2026-10-06T14:32:10Z	ISO 8601, UTC
C	name	Priya K	2–80 chars
D	email	priya@college.edu	Lower-cased
E	year	second	first / second / third / fourth / other
F	domain	web	technical / web / rd / design / media / pr
G	skills	free text	20–800 chars
H	motivation	free text	40–1200 chars
I	portfolio_url	https://...	Optional; http(s) only
J	consent	TRUE	Must be true to submit
K	email_status	sent	pending / sent / failed (Option A only)
L	domain_answer	free text	Answer to the domain-specific question on the application page, 20–800 chars

The website **never writes** to columns to the right of L. Admins can add their own working columns there (e.g. `status`, `reviewer`, `interview_slot`, `notes`) without being affected by the website.

Formula injection: any value starting with `=`, `+`, `-` or `@` is prefixed with `'` before it's written, so a submitted answer can't run as a spreadsheet formula.

Confirmation email (every applicant)

- **When:** sent to every applicant, but only after their row is saved. Never sent for a submission that failed.
- **To:** the email address the applicant entered. **From:** the club address (e.g. `andropedia@<college-domain>`); **Reply-To:** the club inbox.
- **Contains:** applicant's name, reference ID, chosen domain, a short summary of their answers, what happens next (timeline, how shortlisting is communicated), and a contact for questions.
- **Format:** plain-text + simple HTML version; no tracking pixels.
- **On failure:** the application is still accepted (the reference is shown on screen) and the row is marked `email_status = failed`. A retry script (`npm run recruitment:resend-failed`) resends to every `failed` row.
- **Abuse protection:** only one application is accepted per email address (see Spam & abuse), so the form can't be used to send repeated emails to someone else's inbox.

Sending service (choose by expected volume; daily limits are approximate, check current limits):

Service	Approx. free limit	Setup
Resend	~100/day, ~3,000/month	Verify a sending domain for a custom "From"
Gmail SMTP (club Gmail + app password, via Nodemailer)	~500/day (personal Gmail)	Easiest; sends from the club Gmail
Google Workspace account (if college provides)	~2,000/day	Best for large recruitment drives

If a recruitment drive could get more applications in one day than the limit, emails over the limit are queued as `failed` and the retry script sends them the next day.

Spam & abuse

- Hidden spam-trap field. If a bot fills it, the server pretends it succeeded but saves nothing.
- Rate limit: 5 submissions per IP per hour.
- Add Cloudflare Turnstile (a free CAPTCHA) only if spam actually shows up.
- **One application per email address.** Before saving, the server checks the `email` column (D) of the sheet; if the address (compared case-insensitively) is already there, it responds `409` with "An application with this email has already been submitted" and saves and sends nothing. Simultaneous submissions of the same email on one server are blocked in memory. Known limits: two submissions landing on *different* serverless instances in the same instant could both get through, so the dashboard should still treat the earliest row per email as the real one; `name+tag@gmail.com` and dotted Gmail variants count as different addresses.

Configuration (environment variables)

```
# Option A – Sheets API
GOOGLE_SERVICE_ACCOUNT_EMAIL=
GOOGLE_PRIVATE_KEY=
RECRUITMENT_SHEET_ID=
RECRUITMENT_SHEET_TAB=Applications

# Option B – Google Form (instead of the four above)
RECRUITMENT_FORM_ID=

# Email (one of)
RESEND_API_KEY=
SMTP_HOST=smtp.gmail.com
SMTP_USER=
SMTP_PASS=          # Gmail app password, never the account password
EMAIL_FROM="Andropedia <andropedia@...>"
EMAIL_REPLY_TO=
```

Acceptance criteria

- ☐ Submitting the `/join` form adds exactly one row to the sheet, matching the column contract.
- ☐ If the sheet write fails, the applicant sees an error and **no** success screen or email.
- ☐ Every successful application gets a confirmation email with its reference ID, or the row is marked `failed` and the retry script sends it.
- ☐ Downloading the sheet as CSV gives the columns above, in order, with a header row.
- ☐ The sheet is shared only with admins and the service account. It is never public.
- ☐ Bot submissions caught by the spam trap or rate limits create no row and send no email.
- ☐ A second submission with an email already in the sheet (any letter case) is rejected with a clear message, creates no row and sends no email.

2.8 API surface (target)

Method	Route	Access	Notes
POST	/api/recruitment	Public, rate-limited	Validated; adds a row to the Google Sheet; emails the applicant; returns <code>reference</code> (§2.7.1)
GET	/api/me	Logged in	Current user from the session
GET	/api/members	Public	Public fields only, no emails
GET	/api/weeks	Public	
POST/PATCH	/api/admin/weeks[:id]	Super admin	Create, edit, open/close
GET	/api/tasks	Logged in	Scoped by role
POST	/api/tasks	Member	<code>userId</code> from session; week must be open; one per week
POST	/api/evaluations	Lead (own domain) / super admin	Creates or updates; audit-logged
GET	/api/leaderboard	Public	Real <code>domain</code> + <code>period</code> filters
POST	/api/events/:id/rsvp	Public, rate-limited	Capacity + duplicate checks
PATCH	/api/admin/users/:id	Super admin	Role, domain, active

2.9 Non-functional requirements

- **Security:** server-side session check on every non-public route; no secrets in git; Supabase service-role key used only on the server; Row Level Security on tables if the Supabase client is ever used directly from the browser.
- **Privacy:** emails and application contents visible only to authorised admins. The recruitment sheet is shared with named admins only, never "anyone with the link". The Google service-account key and SMTP password live only in environment variables. Applicants are told how their data is used (consent checkbox already exists on `ui-updated`).
- **Reliability:** no user-facing success message unless the write was confirmed by the database.
- **Performance:** leaderboard query < 300 ms for a few hundred members; cache public GETs for 60 s.
- **Ops:** `.env.example`, a README setup section, preview deployments use a separate Supabase project or branch.

2.10 Rollout plan

Phase	Contents	Rough size
0 — Unblock	Decide <code>main</code> vs <code>ui-updated</code> as the base; create the recruitment Google Sheet + service account; pick an email provider; create the Supabase project; share env vars with maintainers privately	½ day
1 — Recruitment	R3: form → Google Sheet + confirmation email + retry script (§2.7.1). Needs no database or auth work.	1–2 days
2 — Stop data loss	R1, R2 (Supabase + Prisma replace <code>data-store.ts</code>)	2–3 days
3 — Lock it down	R4, R5, R6, R7	3–4 days

Phase	Contents	Rough size
4 — Admin & correctness	R8–R10, R12, R13	3–4 days
5 — Extras	R14–R18	as time allows

Recruitment goes first because it's the only problem that loses real people's data, and with the Google Sheet approach it doesn't have to wait for the database.

2.11 Success metrics

- 100% of recruitment submissions appear as a row in the Google Sheet (0 lost)
- 100% of applicants receive a confirmation email (sent straight away, or by the retry script within 24 h)
- 0 protected API routes callable without a valid session (verified by tests)
- 0 public responses containing member emails
- Leaderboard totals match the sum of `Evaluation.score` for every member
- Admins can open a new sprint week without a code deploy

2.12 Open questions

1. Does a Supabase project already exist? (The commit "Configure Prisma for Supabase" suggests someone started one.) Who owns it?
2. Which branch is the base going forward: `main` or `ui-updated`? `ui-updated` has the better recruitment flow but a different folder layout.
3. Login method: magic link only, or also Google sign-in with the college Google account?
4. Is the Google Sheet still where the member list is officially kept, or should the DB replace it after import?
5. Recruitment storage: **Sheets API (recommended)** or **Google Form**? Who owns the Google account that holds the sheet?
6. Roughly how many applications are expected per drive, and the most in a single day? This decides the email provider.
7. Which address should confirmation emails come from: a club Gmail, a college Workspace address, or a custom domain?
8. What should the confirmation email promise about next steps (shortlist date, interview process)?
9. Should admins also get an email for each new application, or is the dashboard enough?
10. How long are applications kept in the sheet after a recruitment cycle ends, and who deletes them?